AutoPark

Smart Parking Management System

Project Report

1. Problem Statement & Solution Approach

The Problem

Traditional parking management systems are plagued by inefficiencies that frustrate both users and administrators. Without real-time visibility into slot availability, drivers spend an average of 15 to 20 minutes circling parking areas, leading to congestion, wasted fuel, and poor user experience.

The core issues break down into five categories:

- **Manual slot tracking** : Staff-based systems introduce human error, cause long queues, and fail to optimize space. A motorcycle may occupy a truck-sized bay while compact spots remain empty.
- **No real-time visibility** : Neither users nor administrators can see the current state of the lot, leading to overcrowding in some areas and waste in others.
- **Walk-in only** : Without digital booking, capacity planning becomes impossible and demand spikes result in chaos.
- **Manual payment** : Paper tickets are prone to fraud, lack audit trails, and provide no analytics for decision-making.
- **No notifications** : Users receive zero feedback about booking confirmations, vehicle entry/exit events, or payment status.

The Solution

AutoPark is a full-stack smart parking management system designed to address all of these pain points. It combines intelligent slot allocation, real-time updates, digital booking, and multi-channel notifications into a single cohesive platform.

- Real-Time Slot Tracking - A live visual parking map shows slot availability across multiple floors, updated instantly via WebSocket. Slots are colour-coded: green for available, red for occupied, yellow for reserved, and grey for maintenance.
- **Intelligent Slot Allocation (Strategy Pattern)** - Users choose between three allocation strategies at booking time: Nearest Slot (minimises walking distance), Cheapest Slot (optimises for budget), or Priority Slot (balances distance and floor level). The system automatically finds the best compatible slot using the chosen algorithm.
- **Vehicle-Aware Compatibility (Factory Pattern)** - The system understands that a Truck can only fit in LARGE slots, a Motorcycle fits in COMPACT, REGULAR, or LARGE, and an Electric Vehicle prefers ELECTRIC_CHARGING slots. The Factory Pattern creates the correct vehicle object and polymorphism determines slot compatibility automatically.
- **Digital Booking Lifecycle** - A complete, trackable workflow: Book → Confirm → Check-In → Active → Check-Out → Complete, with overlap detection to prevent double-bookings.
- **Multi-Channel Notifications (Observer Pattern)** - Booking confirmations, vehicle entry/exit events, and payment receipts are pushed simultaneously to in-app notifications, real-time WebSocket popups, and email.
- **Admin Analytics Dashboard** - Occupancy rates, revenue reports, peak hour analysis, and popular slot rankings give administrators the data they need to make informed decisions.
- **JWT-Based Security** - Access and refresh token authentication with role-based authorisation distinguishes regular users from administrators.

2. Tech Stack

AutoPark is built on a modern, type-safe stack chosen for reliability, developer experience, and scalability. Every layer - from the database to the browser - uses TypeScript, ensuring consistent type safety throughout.

Layer	Technology	Purpose
Frontend	Next.js 15 (App Router)	Server-side rendering, file-based routing
UI Framework	React 19	Component-based UI
Styling	Tailwind CSS	Utility-first CSS framework
Charts	Recharts	Admin dashboard visualisations
Icons	Lucide React	Consistent iconography
Backend	Node.js + Express	REST API server
Language	TypeScript	Type-safe backend and frontend
Database	PostgreSQL	Relational data storage
ORM	Prisma	Type-safe database queries
Real-time	Socket.IO	WebSocket for live updates
Auth	JWT (jsonwebtoken)	Access + Refresh token auth
Password	bcryptjs	Secure password hashing

3. System Design & Architecture

3.1 High-Level Architecture

AutoPark follows a classic 3-Tier Architecture, separating the system into presentation, application, and data layers. This separation keeps concerns cleanly divided and makes each layer independently scalable.

PRESENTATION TIER - Next.js Frontend (Port 3000)
Landing | Auth | Dashboard | Parking Map | Bookings | Vehicles | Payments | Notifications | Admin

↕ *HTTP REST + WebSocket*

APPLICATION TIER - Express Backend (Port 3001)
Auth | User | Vehicle (Factory) | Parking (Strategy) | Booking | Payment | Notification (Observer) | Report

↕ *Prisma ORM*

DATA TIER - PostgreSQL Database

users | vehicles | parking_floors | parking_slots | bookings | payments | entry_exit_logs | notifications

3.2 Backend Layered Architecture

Every backend module follows a strict four-layer architecture. This keeps HTTP handling, business logic, and database access completely separate from one another.

Layer	Responsibility
Controller	Handles HTTP requests and responses. Extends BaseController for consistent formatting.
Service	Contains all business logic and orchestration. This is where patterns like Factory, Strategy, and Observer are applied.
Repository	Manages data access. Extends BaseRepository<T> and encapsulates all Prisma queries behind clean method calls.
Database	PostgreSQL accessed via Prisma ORM. A single shared connection is managed by the DatabaseConnection singleton.

The response flows back through the same layers in reverse - Repository → Service → Mapper (entity to DTO) → Controller → HTTP Response.

3.3 Key Design Decisions

Every architectural choice in AutoPark was made deliberately, with scalability and maintainability in mind.

Decision	Rationale
Singleton Database Connection	Prevents connection pool exhaustion by ensuring only one PrismaClient instance exists across the entire application.
Repository Pattern	Data access is abstracted behind an interface. Switching from PostgreSQL to another database requires only changing the repository implementations, not the services.
Strategy Pattern for Allocation	New allocation algorithms can be added without modifying any existing code. The algorithm can be swapped at runtime with zero downtime.

Observer for Notifications	Adding a new notification channel such as SMS requires zero changes to the existing notification service. Just create a new observer and attach it.
WebSocket for Real-Time	Eliminates polling. Slot availability updates are pushed to clients the moment they change.
JWT with Refresh Tokens	Stateless authentication scales horizontally. Refresh tokens minimise the frequency of re-authentication.
Zod Validation at API Boundary	Invalid requests are rejected before they ever reach business logic, keeping services clean and predictable.

4. SDLC - Software Development Life Cycle

AutoPark was developed following an Agile methodology with six clearly defined phases. Each phase built on the previous, allowing the team to iterate quickly while maintaining architectural integrity.

Phase 1 - Planning & Requirements

- Identified core stakeholders: Parking Users (book, pay, track) and Administrators (manage, monitor, report)
- Defined 29 API endpoints across 8 backend modules
- Selected the technology stack: Next.js, Express, PostgreSQL, and TypeScript throughout
- Identified design patterns to demonstrate: Singleton, Factory, Strategy, Observer, and Repository

Phase 2 - System Design

- Designed the 3-tier architecture separating presentation, application, and data concerns
- Created the database schema with 8 tables and their relationships
- Designed the layered module structure: Controller → Service → Repository
- Defined abstract classes and interfaces to establish the OOP foundation

Phase 3 - Implementation

Development was split into four sub-phases to manage complexity and ensure each layer was stable before the next was built on top of it.

Sub-Phase	What Was Built
Foundation	Monorepo setup, Prisma schema, DatabaseConnection singleton, core abstractions (interfaces, abstract classes, error hierarchy)
Backend	Auth → User → Vehicle (Factory) → Parking (Strategy) → Booking → Payment → Notification (Observer) → Report
Frontend	Auth pages → Dashboard layout → Parking grid → Booking flow → Vehicle management → Admin analytics
Integration	WebSocket real-time updates, database seed script, unit test suite

Phase 4 - Deployment

- PostgreSQL database hosted on Neon (managed cloud PostgreSQL)
- Frontend ready for Vercel deployment; backend ready for Railway or Render
- All sensitive configuration externalised via environment variables

Phase 5 - Maintenance

The modular architecture ensures that future changes are straightforward. The Strategy and Observer patterns mean new features - such as additional allocation algorithms or notification channels - can be added without touching existing code.

5. OOP Concepts Used

5.1 Encapsulation

Encapsulation is the principle of bundling data and methods together while controlling access to internal state. In AutoPark, this shows up in several key places.

The DatabaseConnection class is the clearest example. Its constructor is private, meaning no external code can call new DatabaseConnection(). The internal PrismaClient is also private, accessible only through the public getClient() method. This design ensures the connection is always managed safely, never exposed to accidental misuse.

Similarly, vehicle properties such as licensePlate and type are declared public readonly - they can be read by anyone, but they cannot be changed after the object is created. Repository classes use protected access for their Prisma client, allowing subclasses to use it while keeping it hidden from external consumers.

```
export class DatabaseConnection {
  private static instance: DatabaseConnection;  // hidden from outside
  private client: PrismaClient;                 // cannot be accessed directly

  private constructor() {                       // cannot be instantiated
externally
    this.client = new PrismaClient();
  }

  public static getInstance(): DatabaseConnection {
    if (!DatabaseConnection.instance) {
      DatabaseConnection.instance = new DatabaseConnection();
    }
    return DatabaseConnection.instance;
  }

  public getClient(): PrismaClient {
    return this.client;  // controlled, read-only access
  }
}
```

5.2 Inheritance

Inheritance allows child classes to share behaviour defined in a parent class, reducing duplication and establishing consistent contracts. AutoPark uses inheritance extensively across four class hierarchies.

Parent	Children	What's Inherited
BaseVehicle	Car, Motorcycle, Truck, ElectricVehicle	canFitInSlot(), calculatePrice()
BaseRepository<T>	7 concrete repositories	findAll(), findById(), create(), update(), delete(), count()
BaseController	7 concrete controllers	sendSuccess(), sendCreated(), sendPaginated()
AppError	NotFoundError, ValidationError, AuthenticationError, and 2 more	statusCode, isOperational, stack trace

5.3 Polymorphism

Polymorphism means that objects of different types respond differently to the same method call, without the caller needing to know which type it's dealing with. In AutoPark, this is one of the most practically useful concepts.

When BookingService calculates a price, it calls vehicle.calculatePrice(basePricePerHour, hours) on whatever vehicle object it has. A Car returns the base price, a Motorcycle returns half that, a Truck doubles it, and an Electric Vehicle adds a 20% premium - all through a single method call on a single interface.

Method Call	What Each Type Returns
vehicle.getCompatibleSlotTypes()	Car → [REGULAR, LARGE] | Motorcycle → [COMPACT, REGULAR, LARGE] | Truck → [LARGE] | EV → [ELECTRIC_CHARGING, REGULAR, LARGE]
vehicle.getSizeMultiplier()	Car → 1.0 | Motorcycle → 0.5 | Truck → 2.0 | ElectricVehicle → 1.2
vehicle.calculatePrice(10, 2)	Car → $20 | Motorcycle → $10 | Truck → $40 | EV → $24
strategy.allocate(slots)	Nearest → closest slot | Cheapest → lowest price | Priority → best weighted score
observer.update(payload)	InApp → saves to database | WebSocket → emits to socket | Email → sends email

5.4 Abstraction

Abstraction means hiding complex implementation details behind a clean, simple interface. In AutoPark, abstract classes and interfaces serve as contracts that define what a component does without specifying how it does it.

Abstraction	What It Hides	What It Exposes
BaseVehicle	Cannot be instantiated directly	canFitInSlot(), calculatePrice()
BaseRepository<T>	Prisma queries, connection management	findAll(), findById(), create(), update(), delete()
IParkingStrategy	How allocation actually works	allocate(slots): Slot - one method only
INotificationObserver	Which channel is used (app/socket/email)	update(payload): void - one method only
BaseController	Express response formatting	sendSuccess(), sendCreated(), sendPaginated()

6. Design Patterns Implemented

6.1 Singleton Pattern - Database Connection

The Singleton pattern ensures that only one instance of a class ever exists and provides a global access point to it. In AutoPark, this pattern is applied to the database connection.

Without it, every repository would create its own PrismaClient, each establishing its own connection pool. This would quickly exhaust the database's connection limit. With the Singleton, all seven repositories share a single managed connection.

```
export class DatabaseConnection {
  private static instance: DatabaseConnection;
  private client: PrismaClient;

  private constructor() {
    this.client = new PrismaClient();
  }

  static getInstance(): DatabaseConnection {
    if (!DatabaseConnection.instance) {
      DatabaseConnection.instance = new DatabaseConnection();
    }
    return DatabaseConnection.instance;
  }
}
```

6.2 Factory Pattern - Vehicle Creation

The Factory pattern defines an interface for creating objects while letting subclasses decide which class to instantiate. In AutoPark, this is used to create the right vehicle object based on user input.

When a booking is made, the system needs to know what slot types a vehicle is compatible with and how its size affects pricing. Rather than scattering this logic across services, the VehicleFactory centralises it. Adding a new vehicle type like Bus only requires creating a new subclass and adding one case to the factory - nothing else changes.

```
export class VehicleFactory {
  static create(type: VehicleType, licensePlate: string): BaseVehicle {
    switch (type) {
      case VehicleType.CAR:        return new Car(licensePlate);
      case VehicleType.MOTORCYCLE: return new Motorcycle(licensePlate);
      case VehicleType.TRUCK:      return new Truck(licensePlate);
      case VehicleType.ELECTRIC:   return new ElectricVehicle(licensePlate);
```

```
      default: throw new ValidationError('Unknown vehicle type');
    }
  }
}
```

6.3 Strategy Pattern - Parking Slot Allocation

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime. This is used in AutoPark to handle the different ways users might want to find a parking spot.

The frontend booking form offers three options: Nearest to Entry, Cheapest Price, and Priority. When the user selects one, the strategy name is sent to the API. The ParkingService swaps its internal algorithm at runtime and delegates allocation to whichever strategy is active. The service itself never needs to know the details of any individual algorithm.

```
// Strategy 1: Prioritise proximity
class NearestSlotStrategy implements IParkingStrategy {
  allocate(slots) {
    return slots.reduce((a, b) => a.distanceFromEntry < b.distanceFromEntry ? a :
b);
  }
}

// Strategy 2: Prioritise cost
class CheapestSlotStrategy implements IParkingStrategy {
  allocate(slots) {
    return slots.reduce((a, b) => a.pricePerHour < b.pricePerHour ? a : b);
  }
}

// Strategy 3: Balance both factors
class PrioritySlotStrategy implements IParkingStrategy {
  allocate(slots) {
    return slots.reduce((a, b) => this.score(a) < this.score(b) ? a : b);
  }
  private score(slot) {
    return slot.distanceFromEntry + Math.abs(slot.floorLevel) * 10;
  }
}
```

6.4 Observer Pattern - Notification System

The Observer pattern defines a one-to-many relationship between objects. When the subject (the NotificationService) changes state, all attached observers are notified automatically. This is used to deliver notifications across multiple channels simultaneously.

When a booking is confirmed, the BookingService calls notificationService.notify(payload). The service then broadcasts this to three observers in parallel: InAppObserver (saves to the database), WebSocketObserver (pushes a real-time popup to the browser), and EmailObserver (sends an email). Adding SMS notifications in future means creating a new observer class and attaching it at startup - nothing else changes.

```
class NotificationService implements INotificationSubject {
  private observers: INotificationObserver[] = [];

  attach(observer: INotificationObserver) { this.observers.push(observer); }

  async notify(payload: NotificationPayload) {
    // All observers run in parallel; one failing does not block others
    await Promise.allSettled(this.observers.map(o => o.update(payload)));
  }
}

// At server startup:
notificationService.attach(new InAppObserver());
notificationService.attach(new WebSocketObserver(io));
notificationService.attach(new EmailObserver());
```

6.5 Repository Pattern - Data Access Abstraction

The Repository pattern mediates between business logic and database access, acting like an in-memory collection of domain objects. In AutoPark, all seven repositories extend a single generic BaseRepository<T> that provides common CRUD operations.

Services never write Prisma queries directly. They call methods like findByUserId() or findOverlapping(), which are defined and implemented in the repository. This means business logic stays clean, and if the underlying database ever needs to change, only the repositories are affected.

```
abstract class BaseRepository<T, CreateDTO, UpdateDTO> {
  protected prisma: PrismaClient;
  protected abstract getModelDelegate(): any;

  async findAll(filters?)  { return this.getModelDelegate().findMany({ where:
filters }); }
  async findById(id)       { return this.getModelDelegate().findUnique({ where: {
id } }); }
  async create(data)       { return this.getModelDelegate().create({ data }); }
  async update(id, data)   { return this.getModelDelegate().update({ where: { id },
data }); }
  async delete(id)         { return this.getModelDelegate().delete({ where: { id }
}); }
}
```

```
// Concrete implementation - adds only what's specific to bookings
class BookingRepository extends BaseRepository<Booking, CreateInput, UpdateInput> {
  protected getModelDelegate() { return this.prisma.booking; }
  async findOverlapping(slotId, startTime, endTime) { /* ... */ }
}
```

7. SOLID Principles

AutoPark was designed with all five SOLID principles in mind. These aren't just theoretical - each principle directly influenced architectural decisions in the codebase.

S - Single Responsibility Principle

Every class in AutoPark has exactly one reason to change. AuthController handles only HTTP request/response for auth endpoints. AuthService contains only authentication business logic. UserRepository handles only database access for the users table. VehicleFactory only creates vehicle subclasses.

The practical benefit: if the booking database schema changes, only BookingRepository and BookingMapper need updating. The controller and service are unaffected.

O - Open/Closed Principle

The system is open for extension and closed for modification. Adding a new vehicle type means creating a new class that extends BaseVehicle and adding one case to the factory - existing vehicle classes, ParkingService, and BookingService are untouched. Adding a new allocation algorithm means creating a new class that implements IParkingStrategy - ParkingService and all existing strategies are untouched. Adding a new notification channel means creating a new observer - NotificationService is untouched.

L - Liskov Substitution Principle

Every subtype in AutoPark is fully substitutable for its base type without breaking anything. ParkingService accepts any IParkingStrategy - whether Nearest, Cheapest, or Priority, the contract is honoured. NotificationService calls update() on any INotificationObserver and gets

the expected behaviour. The error handler middleware reads error.statusCode on any AppError subclass and the correct HTTP status is returned.

I - Interface Segregation Principle

No class is forced to implement methods it doesn't need. IParkingStrategy has a single method: allocate(). INotificationObserver has a single method: update(). INotificationSubject has three methods for managing subscriptions. IRepository<T> defines only generic CRUD. Each interface is focused and minimal.

If IParkingStrategy had also required calculatePrice() and updateSlotStatus(), a new strategy would need to implement pricing and slot management logic that has nothing to do with allocation. The current design avoids this entirely.

D - Dependency Inversion Principle

High-level modules depend on abstractions, not concrete implementations. ParkingService depends on IParkingStrategy, not on any specific strategy class. NotificationService depends on INotificationObserver[], not on InAppObserver or WebSocketObserver directly. BookingService receives its NotificationService dependency via constructor injection - it never creates one itself.

```
// BookingService receives its dependency - it does not create it
class BookingService {
  constructor(private notificationService: NotificationService) { }
  // Never does: this.notificationService = new NotificationService();
}
```

8. Base Classes, Interfaces & Class Relationships

8.1 Core Interfaces

Interfaces define contracts without any implementation. They are the backbone of dependency inversion across the system.

```
// Generic data access contract
interface IRepository<T, CreateDTO, UpdateDTO> {
  findAll(filters?): Promise<T[]>;
  findById(id: string): Promise<T | null>;
  create(data: CreateDTO): Promise<T>;
  update(id: string, data: UpdateDTO): Promise<T>;
  delete(id: string): Promise<void>;
```

```
  count(filters?): Promise<number>;
}

// Slot allocation contract (one method - Interface Segregation)
interface IParkingStrategy {
  readonly name: string;
  allocate(availableSlots: AvailableSlot[]): AvailableSlot | null;
}

// Notification observer contract
interface INotificationObserver {
  update(payload: NotificationPayload): Promise<void>;
}

// Notification subject contract
interface INotificationSubject {
  attach(observer: INotificationObserver): void;
  detach(observer: INotificationObserver): void;
  notify(payload: NotificationPayload): Promise<void>;
}
```

8.2 Abstract Base Classes

Abstract classes sit between interfaces and concrete implementations. They provide shared logic while enforcing a contract that subclasses must fulfil.

```
// Forces subclasses to define their slot compatibility and size
abstract class BaseVehicle {
  abstract getCompatibleSlotTypes(): SlotType[];
  abstract getSizeMultiplier(): number;

  // Shared logic - available to all vehicle types
  canFitInSlot(slotType: SlotType): boolean {
    return this.getCompatibleSlotTypes().includes(slotType);
  }
  calculatePrice(base: number, hours: number): number {
    return base * this.getSizeMultiplier() * hours;
  }
}

// Provides CRUD to all repositories; forces each to specify its model
abstract class BaseRepository<T, CreateDTO, UpdateDTO>
  implements IRepository<T, CreateDTO, UpdateDTO> {
  protected abstract getModelDelegate(): any;
  protected abstract entityName: string;
  // ... CRUD implementations
}
```

8.3 Class Relationships

The relationships between classes in AutoPark follow three standard patterns: inheritance (extends), implementation (implements), and composition (has-a).

Inheritance	BaseVehicle → Car, Motorcycle, Truck, ElectricVehicle | BaseRepository → 6 concrete repos | BaseController → 7 controllers | AppError → NotFoundError, ValidationError, AuthenticationError, AuthorizationError, ConflictError

Implementation	IRepository → BaseRepository | IParkingStrategy → NearestSlotStrategy, CheapestSlotStrategy, PrioritySlotStrategy | INotificationObserver → InAppObserver, WebSocketObserver, EmailObserver | INotificationSubject → NotificationService

Composition	ParkingService uses IParkingStrategy (swappable at runtime) | NotificationService manages INotificationObserver[] | BookingService orchestrates NotificationService + ParkingService + Repositories | VehicleFactory creates BaseVehicle subclasses | All Repositories use DatabaseConnection (singleton)

9. UML Diagrams

This section describes the key UML diagrams for AutoPark. The diagrams can be rendered interactively by visiting mermaid.live and pasting the Mermaid code from the project's /docs folder.

9.1 Class Diagram

The class diagram captures the full inheritance hierarchy, interface implementations, and compositional relationships across all modules. Key elements include the BaseVehicle hierarchy (4 concrete types), the BaseRepository hierarchy (7 repositories), the IParkingStrategy implementations (3 strategies), the INotificationObserver implementations (3 observers), and the DatabaseConnection singleton at the root of all data access.

9.2 Use Case Diagram

The use case diagram maps the system's 29 use cases across three actors: User, Admin, and System.

User actions include registering an account, logging in and out, registering vehicles, viewing the parking map, creating bookings (with strategy selection), checking in and out, making payments, viewing history, managing notifications, and cancelling bookings.

Admin actions include accessing the dashboard, managing floors and slots, managing users and roles, viewing revenue and occupancy reports, analysing peak hours, processing refunds, and reviewing all bookings.

System actions include slot allocation via the Strategy pattern, price calculation via the Factory pattern, notification dispatch via the Observer pattern, slot status updates, entry/exit logging, and JWT validation.

9.3 Sequence Diagram - Create Booking Workflow

The sequence diagram for booking creation illustrates how the system coordinates nine components to handle a single user action. The key flow is:

1. User submits booking form with vehicle, strategy preference, and time range
2. BookingService fetches the vehicle record and creates a vehicle object via VehicleFactory
3. Polymorphism: vehicle.getCompatibleSlotTypes() returns the correct slot types for that vehicle
4. ParkingService fetches available compatible slots and delegates to the chosen IParkingStrategy
5. BookingRepository checks for time overlaps on the selected slot
6. Price is calculated via vehicle.calculatePrice() using the size multiplier
7. Booking is saved, slot status is updated to RESERVED
8. NotificationService broadcasts to all three observers in parallel (in-app, WebSocket, email)
9. 201 response returned to frontend; user sees confirmation

9.4 ER Diagram

The database contains eight tables organised around the central Bookings entity. The key relationships are:

Relationship	Cardinality	Description
Users → Vehicles	1 : N	One user owns many vehicles
Users → Bookings	1 : N	One user makes many bookings over time
ParkingFloors → ParkingSlots	1 : N	One floor contains many slots
ParkingSlots → Bookings	1 : N	One slot can have many bookings across different time periods
Bookings → Payments	1 : 1	Each booking has exactly one payment record
Bookings → EntryExitLogs	1 : 1	Each booking has exactly one entry/exit log
Users → Notifications	1 : N	One user receives many notifications

Thank You